

# Manual Técnico

## Practica1

José Fernando Calderón Castillo 202330678

## Introducción

Nos basaremos en los videojuegos de la primera generación: Pokemon Rojo, Azul y Amarillo, sin embargo, por fines prácticos no será una copia exacta del juego, habrán ciertas modificaciones para adecuar el proyecto a los requerimientos del curso.

En cuanto a la interfaz, la aplicación será completamente en consola. Se deja a creatividad y criterio del desarrollador la forma de presentar el juego en consola, para ello se permite el uso de caracteres ASCII y códigos ANSI.

Al iniciar el sistema se presenta al usuario una breve explicación del juego y se le dan las opciones:

- Iniciar una partida nueva: Inicializa una partida limpia, el flujo del juego empieza desde cero, generando nuevos mapas y elementos.
- Continuar una de las partidas previamente guardadas: Despliega la lista de partidas guardadas disponibles en el sistema. Al seleccionar una, el programa restaura el progreso del jugador y todos los elementos de la partida seleccionada y continúa el flujo del juego. Para el usuario no debe haber cambio aparente en la partida, es decir, los datos deben ser exactamente como eran al momento de guardar la partida.
- Cerrar el juego: Termina la ejecución de la aplicación.

## Tecnologías utilizadas

En este proyecto se utilizó la versión de Java JDK 21 la cual dejó el link de descarga:

<https://www.oracle.com/java/technologies/javase/jdk21-archive-downloads.html> El IDE

utilizado fue Netbeans 25 el cual dejó el link de descarga aquí:

[https://netbeans-apache-org.translate.goog/front/main/download/nb25/?\\_x\\_tr\\_sl=en&\\_x\\_tr\\_tl=es&\\_x\\_tr\\_hl=es&\\_x\\_tr\\_pto=tc](https://netbeans-apache-org.translate.goog/front/main/download/nb25/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc)

El sistema de construcción fue Maven

## Algoritmos principales / diseño del programa de la practica 1

Pseudo código del menú:

Inicio Verificar Victoria

Inicio Si (Jugador tiene 3 medallas)

Retorna verdadero

Escribir "Has ganado"

Sino

Retornar Falso

Fin Si

Fin

Inicio turno Jugador

Escribir "Atacar

Cambiar Pokémon

Usar objeto

Lanzar Pokémon

Huir "

Leer Opción

Inicio Switch (Opción)

Caso 1 → Llamar Atacar

Caso 2 → Llamar Cambiar Pokémon

Caso 3 → Llamar Machetear

Caso 4 → Llamar Huir

Caso 5 → "Opción inválida"

Fin Switch.

Fin.

Inicio Capturar

Inicio si (No (Puede ser capturado))  
retornar Falso

Fin si

Generar num Aleatorio

Inicio si (Numero Aleatorio cumple condicion de captura  
agregar Pokemon [i]  
retornar verdadero

Sino

retornar falso

Fin si

Fin Capturar

Inicio turno Oponente

Inicio si (Existe Pokemon disponible)

llamar elegir Ataque

llamar ejecutar Ataque

llamar dano

Fin si

Fin turno Oponente

Inicio ~~Hacer~~ Recibir Daño

Salud Actual  $<$  Salud Actual - Daño

Inicio Si (Salud Actual  $\leq 0$ )

Salud Actual = 0

Escribir "El Pokemon esta debilitado"

Fin Si

Fin

(Inicio restaurar Salud)

Salud Igual

Inicio restaurar Salud

Salud Actual == Salud Maxima

Estado == Normal

Fin restaurar Salud.

Inicio menu.

Escribir { NuevaPartida  
Cargar Partida  
Salir }

Leer opción

Inicio switch

Caso 1: llamar nuevaPartida();

Caso 2: llamar CargarPartida();

Caso 3: ~~return~~ Breach.

Salir al menu.

Fin switch.

Fin menu.

Inicio mover Jugador

Var movimiento

Escribir "A, S, D, W, Es Para moverse"

Leer movimiento

Var Nueva Fila

Var Nueva Columna.

Inicio Switch

Caso "W" → NuevaFila - 1;

Caso "S" → NuevaFila + 1;

Caso "A" → NuevaColumna - 1;

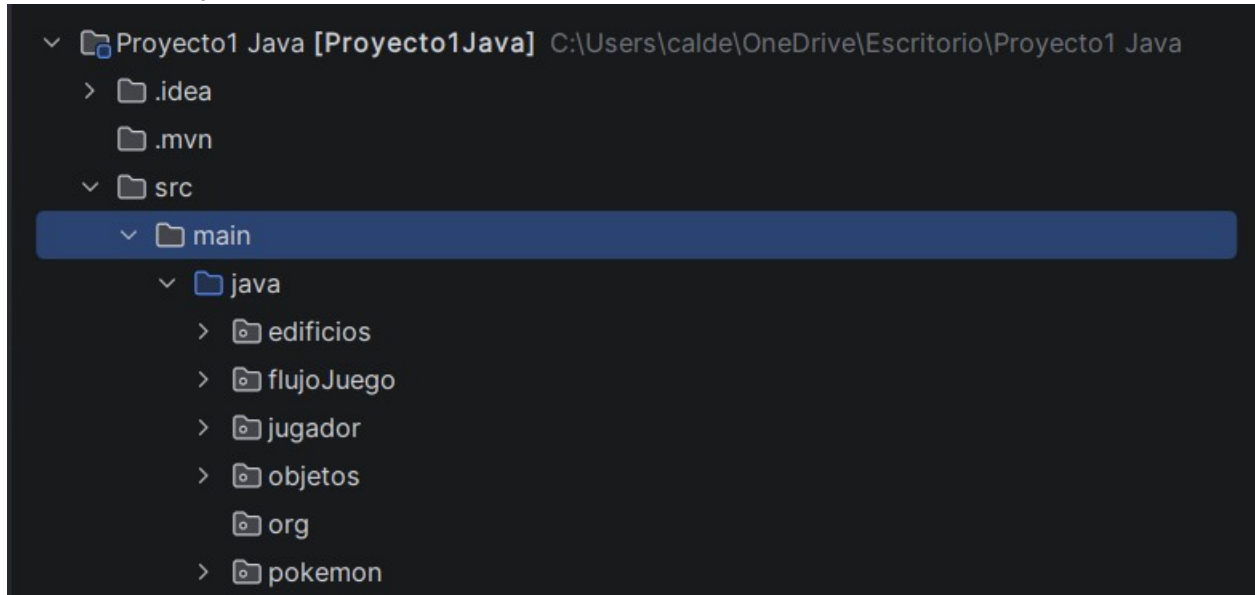
Caso "D" → NuevaColumna + 1;

Fin

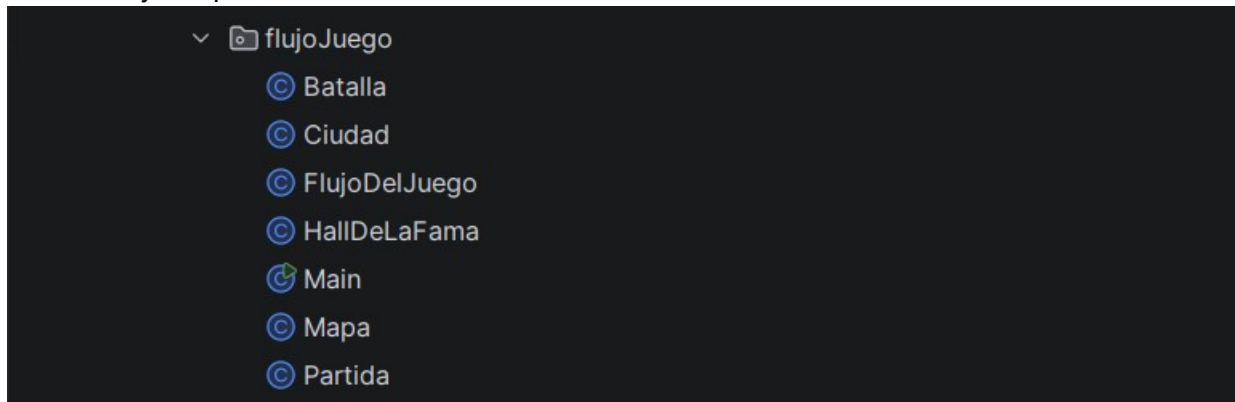
Fin

## Datos técnicos de la practica:

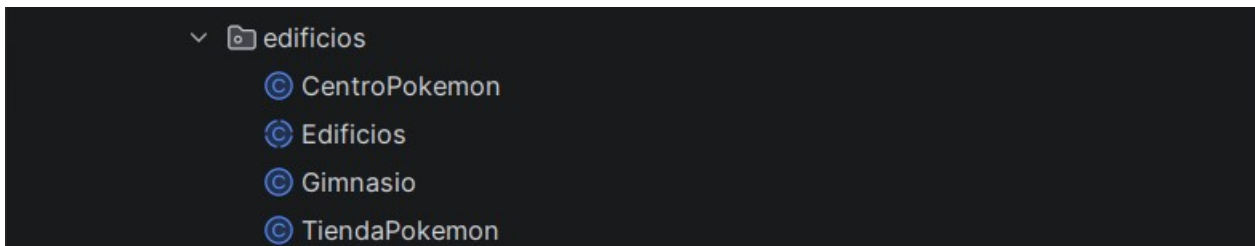
Estructura del proyecto:



Debido a que usamos Programación Orientada a Objetos tenemos que usar carpetas para poder separar los objetos por características familiares.



La carpeta de flujo de juego es la encargada de la logica detras del juego



La carpeta de los edificios es la que esta encargada de la generación de los espacios interactivos

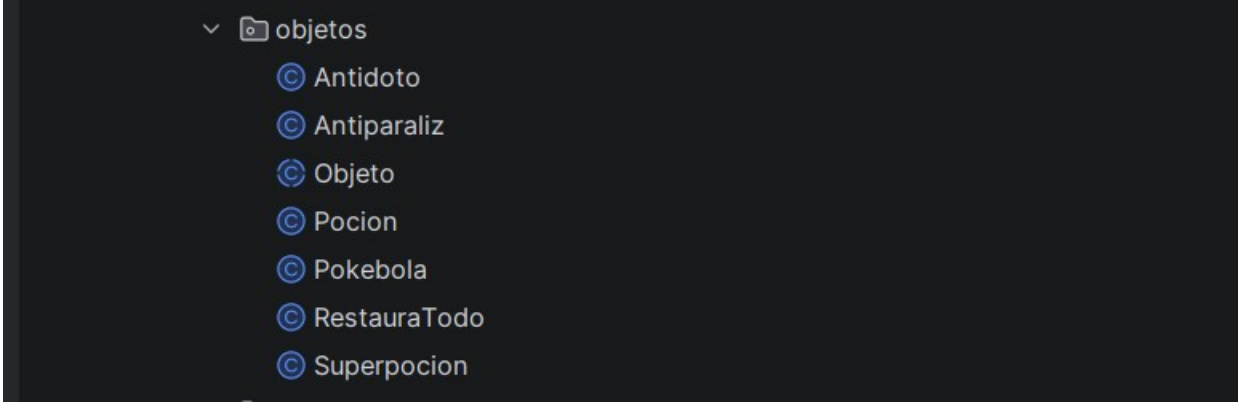


```

  v  [icon] jugador
      [icon] Jugador
      [icon] Mochila

```

La carpeta de jugador es la encargada de asignarle al usuario todos los objetos y características

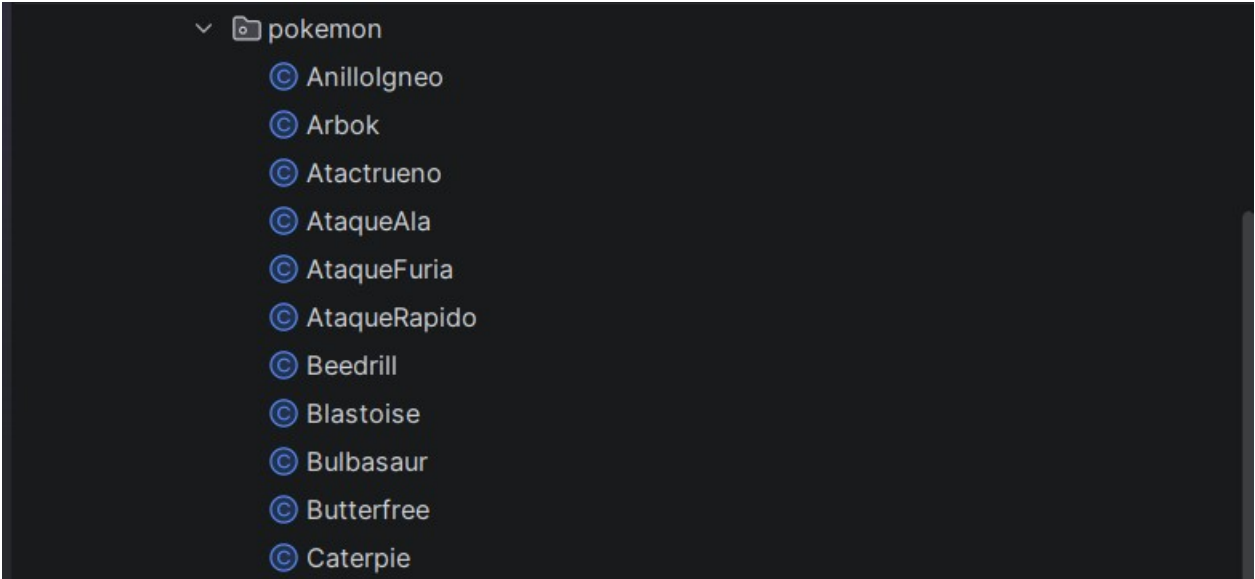


```

  v  [icon] objetos
      [icon] Antidoto
      [icon] Antiparaliz
      [icon] Objeto
      [icon] Pocion
      [icon] Pokebola
      [icon] RestauraTodo
      [icon] Superpocion

```

La carpeta de los objetos esta encargada de almacenar todos los objetos que se pueden utilizar



```

  v  [icon] pokemon
      [icon] AnilloIgneo
      [icon] Arbok
      [icon] Atactrueno
      [icon] AtaqueAla
      [icon] AtaqueFuria
      [icon] AtaqueRapido
      [icon] Beedrill
      [icon] Blastoise
      [icon] Bulbasaur
      [icon] Butterfree
      [icon] Caterpie

```

Luego tenemos a la carpeta Pokemon que es la encargada de almacenar todas las especies de pokemon y ataques disponibles

## Explicación del código

```
1 package flujoJuego;
2
3 // El main lo dejo solo para llamar al Flujo del juego
4 public class Main {
5     public static void main(String[] args) {
6
7         // Se crea el objeto Flujo de juego y se guarda en la variable juego
8         FlujoDelJuego juego = new FlujoDelJuego();
9
10        // llamamos al sub metodo menu principal
11        juego.menuPrincipal();
12    }
13 }
```

Debido a que estamos usando POO. El main debe no debe estar cargado por lo cual su unico proposito es llamar al metodo que controla la logica del juego

```
1 package flujoJuego;
2
3 import java.util.Scanner;
4
5 // se hace publico para que el main pueda acceder
6 public class FlujoDelJuego { 2 usages
7
8     // se declaran los atributos
9     private Partida partida; 3 usages
10    private Scanner scanner; 3 usages
11
12    // se crea un constructor
13    protected FlujoDelJuego() { 1 usage
14        // this se usa para decir que el atributo es del objeto (basicamente se instancia)
15        this.partida = new Partida();
16        this.scanner = new Scanner(System.in);
17    }
```

Aqui pasan dos cosas importantes primero nos indica en que carpeta nos encontramos por eso dice package flujoJuego, lo que nos dice que se encuentra en la carpeta flujoJuego

y la clase principal se hace publica así cualquier otra clase puede acceder a esta

sus atributos son privados debido a que solo esta clase las usara, y el constructor es protected debido que otra clase dentro del mismo paquete la va a utilizar

```

41 // Aquí llama a los metodos dependiendo la eleccion
42 switch (opcionSalida) {
43
44     case 1:
45         partida.nuevaPartida();
46         break;
47
48     case 2:
49         partida.cargarPartida();
50         break;
51
52     case 3:
53         System.out.println();
54         System.out.println("Saliendo del juego...");
55         break;
56
57     default:
58         System.out.println();
59         System.out.println("Opcion invalida.");
60         break;
61 }
62 // el ciclo se cierre si la opcion es 3
63 } while (opcionSalida != 3);
64 }
65 }

```

Después de imprimir las opciones las leemos pero esta entra a un switch lo cual llama a los submetodos correspondientes que vienen de otros objetos, y se cierra cuando el jugador presiona 3

```

1 package flujoJuego;
2
3 // ya que en partida esta la logica del juego se importan las otras carpetas para poder usarlas
4
5 // se importa para poder guardar los datos de la partida
6 import java.io.FileInputStream;
7 // se importa para poder leer los datos de la partida
8 import java.io.FileOutputStream;
9 // manejar errores de archivos
10 import java.io.IOException;
11 // cargar archivos
12 import java.io.ObjectInputStream;
13 // Permite guardar objetos
14 import java.io.ObjectOutputStream;
15 // permite que la clase pueda guardarse como objeto
16 import java.io.Serializable;
17 import java.util.Random;
18 import java.util.Scanner;
19 import jugador.Jugador;
20 import objetos.Pocion;
21 import objetos.Pokebola;
22 // aqui se importan todas las clases de la carpeta pokemon
23 import pokemon.*;

```

Importamos varias extensiones de java, por ejemplo FileInputStream la cual se encarga de convertir los objetos a bites, el FileOutputStream, se encarga de leer los bites a datos, IOException esta encargado de manejar errores a la hora de convertir archivos, ObjectInputStram, se encarga de convertir objetos a bites, ObjectOutpuDataStream se encarga de pasar los bites a datos que java puede usar en consola.

Pero tambien importamos las carpeetas que vamos a utilizar pero aquí pasa algo importante y es que importamos toda la carpeta pokemon y no lo hacemos uno en uno por que seria mucho codigo

```

25 // el implements serializable se usa para poder convertir partida en datos
26 public class Partida implements Serializable { 4 usages
27
28 // Basicamente es un indicador de la version del serializador y se usa L porque asi se guarda como 64 bits
29 private static final long serialVersionUID = 1L; no usages
30
31 // se instancian todos los atributos del objeto
32 private String nombrePartida; 6 usages
33 private Jugador jugador; 30 usages
34
35 private transient Scanner scanner; 7 usages
36
37 private Ciudad[] ciudades; 15 usages
38 private Ciudad ciudadActual; 13 usages
39
40 private int batallasSalvajes; 2 usages
41 private int batallasEntrenadores; 1 usage
42 private int pokebolasLanzadas; 1 usage
43 private int pokemonCapturados; 1 usage
44 private int entrenadoresDerrotados; 3 usages
45

```

Aqui pasa algo importante se utiliza la palabra implements serializable ya que así indicamos que este objeto se puede convertir en datos. Pero también utilizamos private static final long lo que significa que hacemos una constante pero esta constante es un indicador de la versión que se esta serializando, y el 1L esta indicando que numero de versión es pero la L indica que es long es decir que guarda datos grandes en lugar de guardar 32 bits guarda 64 bits. Tambien instanciamos a los atributos del objeto, es decir es que tiene el objeto

```

46 // Escaner que se va a usar en esta clase
47 > protected Partida() { this.scanner = new Scanner(System.in); }
48
49 // se usa un getScanner ya que asi obtenemos un valor
50 private Scanner getScanner() { 8 usages
51     if (scanner == null) {
52         scanner = new Scanner(System.in);
53     }
54     return scanner;
55 }
56
57 // Este metodo es para cuando el jugador crea una nueva partida
58 protected void nuevaPartida() { 1 usage
59
60 // se instancian todas las variables que utilizaremos
61 this.batallasSalvajes = 0;
62 this.batallasEntrenadores = 0;
63 this.pokebolasLanzadas = 0;
64 this.pokemonCapturados = 0;
65 this.entrenadoresDerrotados = 0;
66
67 // llamamos al metodo para crear ciudades
68 crearCiudades();
69
70 // ya que nos piden que las ciudades sean aleatorias aqui metemos las ciudades
71 int numeroAleatorio = (int) (Math.random() * ciudades.length);
72 this.ciudadActual = ciudades[numeroAleatorio];
73

```

También utilizamos un geter el cual es encargado de acceder a un atributo privado de una clase y debido a que utilizamos un scanner que utilizaremos durante toda la clase. Tambien usamos

```

59 // Este metodo es para cuando el jugador crea una nueva partida
60 protected void nuevaPartida() { 1 usage
61
62 // se instancian todas las variables que utilizaremos
63 this.batallasSalvajes = 0;
64 this.batallasEntrenadores = 0;
65 this.pokebolasLanzadas = 0;
66 this.pokemonCapturados = 0;
67 this.entrenadoresDerrotados = 0;
68
69 // llamamos al metodo para crear ciudades
70 crearCiudades();
71
72 // ya que nos piden que las ciudades sean aleatorias aqui metemos las ciudades
73 int numeroAleatorio = (int) (Math.random() * ciudades.length);
74 this.ciudadActual = ciudades[numeroAleatorio];
75
76 // llamamos a los submetodos que hacen que el juego funcione
77 bienvenidaOAK();
78 jugarMapa();
79 datosPartida();
80 }

```

dentro de la clase de nueva partida instanciamos los atributos del objeto y llamamos al submetodo encargado de crear ciudades, despues de crear las ciudades que utilizaremos llamamos a los metodos correspondientes

```

84 // este metodo crea las ciudades
85 private void crearCiudades() { 1 usage
86
87 // Aqui se guardan los nombres de las ciudades
88 String[] nombres = {
89     "Pueblo Paleta", "Ciudad Verde", "Ciudad Plateada",
90     "Ciudad Celeste", "Ciudad Carmin"
91 };
92
93 // Ya teniendo las ciudades asignamos cuantas ciudades tienen que ser
94 ciudades = new Ciudad[3];
95
96 // aqui se define cual es la primera ciudad
97 int ciudad1 = (int) (Math.random() * nombres.length);
98 ciudades[0] = new Ciudad(nombres[ciudad1]);
99
100 // La ciudad 2
101 int ciudad2;
102
103 // siempre se hara hasta encontrar un numero distinto a ciudad1 asi no se repiten
104 do {
105     ciudad2 = (int) (Math.random() * nombres.length);
106 } while (nombres[ciudad2].equals(ciudades[0].getNombre()));
107 ciudades[1] = new Ciudad(nombres[ciudad2]);

```

Metodo que crea ciudades, basicamente escoge 1 de 5 ciudades que previamente se definieron en un arreglo y tambien verifica que no se hayan escogido antes, solo puede escoger 3 ciudades

```

166 // y entra en un switch
167 switch (opcion) {
168     case 1 -> {
169         System.out.println("Has elegido a Bulbasaur.");
170         // Una vez instanciado se llama al submetodo agregar jugador y se crea un objeto bulbasaur
171         this.jugador.agregarPokemon(new Bulbasaur());
172     }
173     case 2 -> {
174         System.out.println("Has elegido a Charmander.");
175         this.jugador.agregarPokemon(new Charmander());
176     }
177     case 3 -> {
178         System.out.println("Has elegido a Squirtle.");
179         this.jugador.agregarPokemon(new Squirtle());
180     }
181 }
182 // el ciclo se repite si la opcion es mayor que 3 o menor de 1
183 } while (opcion < 1 || opcion > 3);
184 }

```

Este metodo es el encargado de escoger al pokemon inicial, despues que el jugador escoga algo este llama al submetodo que guarda el pokemon inicial en el equipo del jugador

```

186 // Este metodo es el que llama al mapa
187 protected void jugarMapa() { 2 usages
188
189 // variable que representa al movimiento
190 String movimiento;
191 // Siempre se llamara al mapa y sus condicionales
192 do {
193     System.out.println();
194     // Ya una vez definida la ciudad en la que estamos se llama al mapa
195     ciudadActual.mostrarMapa();
196
197     // Las instrucciones para hacer mas intuitivo el mapa
198     System.out.println("[W/A/S/D] Mover | [1] Equipo | [2] Mochila | [3] Perfil | [4] Cambiar Ciudad | [5] Guardar | [6] Pokedex | [7] Salir");
199     System.out.print("Opción: ");
200     movimiento = getScanner().nextLine().toUpperCase();
201

```

Aqui utilizamos un string debido a que no solo usamos numeros si no tambien usamos caracteres

```

// dependiendo de lo que escoga el jugador entra al switch
switch (movimiento) {
    // en caso de que el jugador escoga estas opciones se llama al sub metodo para mover al jugador
    case "W", "S", "A", "D" -> {
        char casilla = ciudadActual.moverJugador(movimiento);
        procesarCasilla(casilla);
    }
    // los otros casos xd
    case "1" -> equipoPokemon();
    case "2" -> mochila();
    case "3" -> perfil();
    case "4" -> cambiarCiudad();
    case "5" -> guardarPartida();
    case "6" -> pokedex();
    case "7" -> System.out.println("Saliendo del mapa...");
    default -> System.out.println("Opción inválida.");
}
} while (!movimiento.equals("7"));
}

```

Este cae a un switch y en caso de que nos movamos llamamos al submetodo encargado de eso y las otras opciones tambien llama a los submetodos correspondientes

```

293 protected void guardarPartida() { 1 usage
294 /*
295 // Basicamente El nombre de Partida se convertira el nombre del archivo donde se guardara
296 // El FileOutputStream Sirve para abrir o crear el archivo donde se van a escribir los datos
297 // ObjectOutputStream es para guardar los objetos en archivos
298 // writeObject basicamente guarda el objeto completo en el archivo de partida
299 */
300 // Gracias al try el ObjectOutputStream se cierra cuando termina de usarse
301 try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(nombrePartida + ".dat"))) {
302     oos.writeObject(this);
303     System.out.println("Partida '" + nombrePartida + "' guardada con éxito");
304     // el catch es para que evitamos errores por en caso de que no haya o no pueda acceder al archivo
305 } catch (IOException e) {
306     System.out.println("Error al guardar la partida: " + e.getMessage());
307 }
308 }

```

Este metodo es importante ya que basicamente El nombre de Partida se convertira el nombre del archivo donde se guardara, el FileOutputStream Sirve para abrir o crear el archivo donde se van a escribir los datos, ObjectOutputStream es para guardar los objetos en archivos y writeObject basicamente guarda el objeto completo en el archivo de partida

El try se encarga que despues de utilizar el ObjectOutputStream se cierre solo y nos ahorremos codigo y el catch es para evitr errores por en caso de que no hayan o no pueda acceder a los archivos

```

310 // este metodo es para cargar la partida
311 protected void cargarPartida() { // usage
312     // esto es importante ya que pedimos el nombre de la partida ya que asi podemos llamar a los datos
313     System.out.print("Escriba el nombre de la partida que desea cargar: ");
314     String nombre = getScanner().nextLine();
315
316     /*
317     // El file input es para poder llamar el archivo previamente creado
318     // El objetInput es para poder leer los objetos previamente guardados
319     // el readObject es para poder leer lo que estaba guardado en los archivos
320     */
321     try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(nombre + ".dat"))) {
322         Partida cargada = (Partida) ois.readObject();
323         // aqui se reescribe el nombre de la partida
324         this.nombrePartida = cargada.nombrePartida;
325         // se reescribe el nombre del jugador
326         this.jugador = cargada.jugador;
327         // se reescribe las ciudades que se habian seleccionado
328         this.ciudades = cargada.ciudades;
329         // se reescribe la ciudad en la que el jugador se encuentra
330         this.ciudadActual = cargada.ciudadActual;
331         // se reescribe los entrenadores derrotados
332         this.entrenadoresDerrotados = cargada.entrenadoresDerrotados;
333         System.out.println("Partida cargada exitosamente.");
334         jugarMapa();

```

El metodo para cargar partidas usa cosas importantes como file input es para poder llamar el archivo previamente creado, objetInput es para poder leer los objetos previamente guardados y el readObject es para poder leer lo que estaba guardado en los archivos.

El try es para poder leer el nombre de partida y agrega el .dat para que java lo pueda leer, despues solo sobre escribe la informacion sobre los atributos actuales

```

36 // Generar mapa
37 protected void generar() { // usage
38
39     // Se imprime el mapa con espacios basillos
40     for (int i = 0; i < filas; i++) {
41         for (int j = 0; j < columnas; j++) {
42             mapa[i][j] = ' ';
43         }
44     }
45
46     // Muros verticales
47     for (int columna = 0; columna < columnas; columna++) {
48         mapa[0][columna] = 'Y';
49         mapa[filas - 1][columna] = 'Y';
50     }
51
52     // Muros Horizontales
53     for (int fila = 0; fila < filas; fila++) {
54         mapa[fila][0] = 'Y';
55         mapa[fila][columnas - 1] = 'Y';
56     }

```

Aqui se crea el mapa ya que tenemos un ciclo for hacemos que este imprima espacios en blancos, seguido hacemos que se impriman los vordes verticales y horizontales

```
58 // Posición inicial
59 jugadorFila = 5;
60 jugadorColumna = 3;
61 elementoDebajoJugador = ' ';
62 mapa[jugadorFila][jugadorColumna] = 'J';
63
64 // Elementos interactivos
65 colocarElemento('C');
66 colocarElemento('T');
67 colocarElemento('G');
68 colocarHierba();
69 colocarRocas();
70 }
```

Yo definí la posición del jugador y llamamos al método para poder colocar las casillas especiales

## **Instalación del entorno de desarrollo**

Para instalar el archivo es necesario el uso de un IDE. Como NeatBeans ya que hay una carpeta llamada Practica1 que también contiene el manual técnico, manual de usuario y la carpeta del archivo.

En este caso para ejecutarlo hay que acceder a consola y escribir este comando `java -jar Proyecto1.jar` el cual ejecutara el juego pero si quieres acceder al código fuente hay que importar toda la carpeta, luego ya solo ver que parte del código se quiere